

Week 12: File Systems

TABLE OF CONTENTS

Complete Table of Contents

- 1 [Lab 4 Reviews](#)
- 2 [File System Introduction](#)
 - a [What is a File System? What does it do?](#)
 - b [Directory vs. File](#)
 - c [A Simplified File System Diagram](#)
 - d [Reading a File \(`open\(\)` , `read\(\)` \)](#)
 - e [An Aside: More Details](#)
 - a [Inodes](#)
 - b [Multi-Level Index to Data Blocks](#)
 - c [Disk Layout](#)
 - f [Writing a File \(`write\(\)` \)](#)
 - g [Deleting a File: `unlink\("/foo/bar"\)`](#)
 - a [Step 1: Traverse the path \(same as open\)](#)
 - b [Step 2: Decrement the link count](#)
 - c [Step 3: Free the data blocks](#)
 - d [Step 4: Free the inode](#)
 - e [Step 5: Remove the directory entry](#)
 - f [Full I/O timeline](#)
 - g [Key things to notice](#)
 - h [More Details](#)
 - a [Links](#)
 - i [Tying Everything to Lab 5](#)
 - j [Next Week: More File Systems](#)

Lab 4 Reviews

File System Introduction

My plan for introducing File Systems:

- This week, focus on how a FS such as the one we're implementing for Lab 5 does the common operations (`read()` , `open()` , `write()`)
- Introduce some core concepts around the implementation
- Next week, talk about FSs at higher level, providing explanations and alternatives for details we see today

A file system provides:

- Persistence: Data doesn't go away
- Naming: Human-Friendly directory trees
- Mapping: Logical files to physical blocks

What is a File System? What does it do?

- From the user's perspective:
 - Translate a filename to an address on a disk where the data lives
- From the system's perspective:

File Systems:

- Persistence

- Naming

- Mapping

filename



data

• filename



disk blocks

- Block size 4kB

- 100G $\Rightarrow$ 25.6M

Directory vs. File

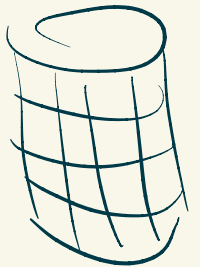
filename1 - 64

3 operations

- read()
- write()
- delete()

Data
Bitmap

Inode
Bitmap



Data
Blocks

Inode
List

- A bunch of *disk blocks*
- The big idea behind File Systems: **Translate a Filename and Offset into a Block number**
- What's a Block number?
 - We take our storage device (e.g. hard drive) and break it into a bunch of Blocks
 - A typical block size might be 4KB
 - If our drive is 100G, that gives us 25.6 million blocks
 - The block number indicates a span of bytes on a disk that can be used to store data

Directory vs. File

A file:

- The data of a file is stored in a block on the disk
- The file system keeps track of files and which blocks are used to store the data for the file
- Things we can do with files:
 - `read()`
 - `write()`
 - `delete()`

A directory:

- Is a special kind of file
- Maintains the mapping between a *filename* and a *file number*
 - Makes storing files human-friendly!
 - Allows us to put some structure in place to make it easy for humans to think about our files
- Even though directories are just files, the file system exposes different functionality
 - e.g. We create a directory (e.g. via `mkdir()`) rather than `write()` a file
 - This allows us to protect the directory structure and prevents something from (for example) corrupting the filename->file number mappings.

A Simplified File System Diagram

Components of a File System

- Data Bitmap: Keeps track of what data blocks are free or occupied
- Inode Bitmap: Keeps track of what Inode blocks are free or occupied
 - In short, **Index Nodes**: think about them as a data structure that holds details about every file
 - Every file has its own inode!
 - We refer to inodes by their inumber: If we know a file's file number, we know its inumber and can find the inode, which gives us data about the file.
- Inode List: A list of all the inodes and their data
- Data Blocks: Where data gets stored on the disk

Reading a File (`open()`, `read()`)

Goal: Read file `/foo/bar/baz.txt`

In C, this would look like this:

```
#include <stdio.h>

int main() {
    FILE *f = fopen("/foo/bar/baz.txt", "r");
    if (f == NULL) {
        perror("fopen");
        return 1;
    }

    char buf[4096];
```



```

    size_t n;
    while ((n = fread(buf, 1, sizeof(buf), f)) > 0) {
        fwrite(buf, 1, n, stdout);
    }

    if (ferror(f)) {
        perror("fread");
    }

    fclose(f);
    return 0;
}

```

What the FS does to make this happen:

- 1 Start by visiting the `/` directory
 - a Read `/` inode
 - a This is the **root inode**, a special inode with a i-number
 - b Find the block that stores the filename/finenumber mapping
 - c Read the data in that block to get `foo`'s finenumber
 - How do we know which block contains the mapping for `foo`?
- 2 Visit the `foo` directory
 - a Go to the inode for `foo`'s finenumber
 - b Read the data block to get the between `bar` and it's file number
- 3 Visit the `bar` directory
 - a Go to the inode for `bar`
 - b Get the data block and read the data to get the finenumber for `baz`
- 4 Visit the file `baz`
 - a Go to the inode for `baz` (since we know the finenumber)
 - b Iterate through all the data blocks:
 - a Go to the data block "on disk" and read the data there

Questions to answer:

- How many READ operations do we do?
- Are there ever any WRITE operations?
- How many inodes do we visit?

10 Reads (assume 3 blocks of data)
4 inodes: /, foo, bar, baz.txt

An Aside: More Details

Inodes

An **inode** is a data structure that holds metadata for files in a file system. The details vary from FS to FS, but there's a common structure that holds even when details might vary.

Here's an example inode data structure:

Size (bytes)	Field name	Purpose
2	mode	Read/write/execute permissions
2	uid	Owner's user ID
4	size	File size in bytes
4	atime	Last access time
4	ctime	Last inode change time
4	mtime	Last modification time
4	dtime	Deletion time
2	gid	Group ID of file's group
2	links_count	Number of hard links to this file
4	blocks	Number of disk blocks allocated
4	flags	ext2 behavior flags
4	osd1	OS-dependent field
48	block	Disk block pointers (10 direct, 1 single-indirect, 1 double-indirect)
4	generation	File version number (used by NFS)
4	file_acl	File access control list
4	dir_acl	Directory access control list

This inode has:

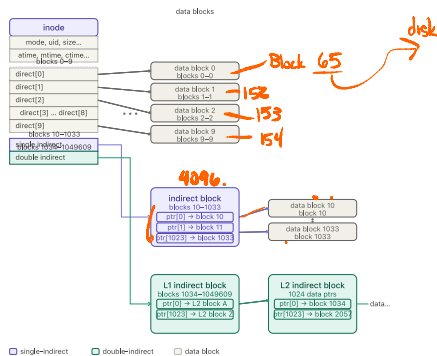
- 10 direct pointers (4 bytes each = 40 bytes)
- 1 single-indirect (4 bytes)
- 1 double-indirect (4 bytes)

Multi-Level Index to Data Blocks

Similar to what we saw with Virtual Memory, if the file has more than 10 blocks worth of data, we utilize a multi-level tree structure to keep track of all the data blocks.

The block ranges shown assume 4 KB blocks and 4-byte pointers (so 1024 pointers per indirect block):

- **Direct pointers** cover blocks 0–9 (10 blocks = 40 KB)
- **Single-indirect** covers blocks 10–1033 (1024 blocks = 4 MB)
- **Double-indirect** covers blocks 1034–1,049,609 (1024 × 1024 = 1,048,576 blocks ≈ 4 GB)



(Note: Image created by Claude)

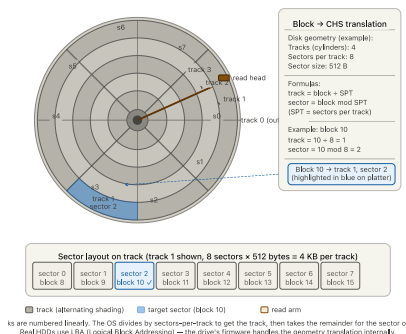
Looking up blocks in a file:

- To look up block 500 of a file, the FS knows it falls in the single-indirect range ($500 > 9$, $500 < 1034$), subtracts the 10 direct blocks to get index 490, reads the indirect block, and follows **ptr[490]** to the data.
- For block 2000, it's in the double-indirect range — subtract 1034 to get index 966, divide by 1024 to find which L2 block (index 0), and use the remainder (966) as the offset into that L2 block.

Disk Layout

In the previous section, we said “the FS follows **ptr[490]** to the data”. What does that mean?

- Since this is similar to what we talked about with Virtual Memory, it kinda feels like the data in **ptr[490]** is a memory address (e.g. 0x281938aff).
- BUT: Remember, we’re working with a persistent data store, so a disk drive of some kind. Is that the same?
 - Not really!



(Note: Image created by Claude)

This diagram shows an example of how data block numbers might translate to an actual location on the disk (in this case, the HDD).

- With the FS, once we look up the block number that holds the data, we have to go to that block to find the data.
 - This might look different for different kinds of drives.
 - Specifically: For our Lab 5, our drive is in memory— so the translation has to be from block number → memory location.
- To be clear: **Modern drives don’t actually expose this geometry.**
 - They use LBA (Logical Block Addressing), where the host just sends a linear block number and the drive’s own firmware translates it to physical track/sector/head internally.
 - This decouples the OS from physical geometry details, which is important since modern drives have variable sectors-per-track (outer tracks fit more sectors than inner ones) and complex remapping for bad sectors.

Writing a File (write)

Goal: Create a new file `/foo/neu.txt` (Assume directory `foo` already exists).

The C code to do this:

```
#include <stdio.h>

int main() {
    FILE *f = fopen("/foo/neu.txt", "w");
    if (f == NULL) {
        perror("fopen");
        return 1;
    }

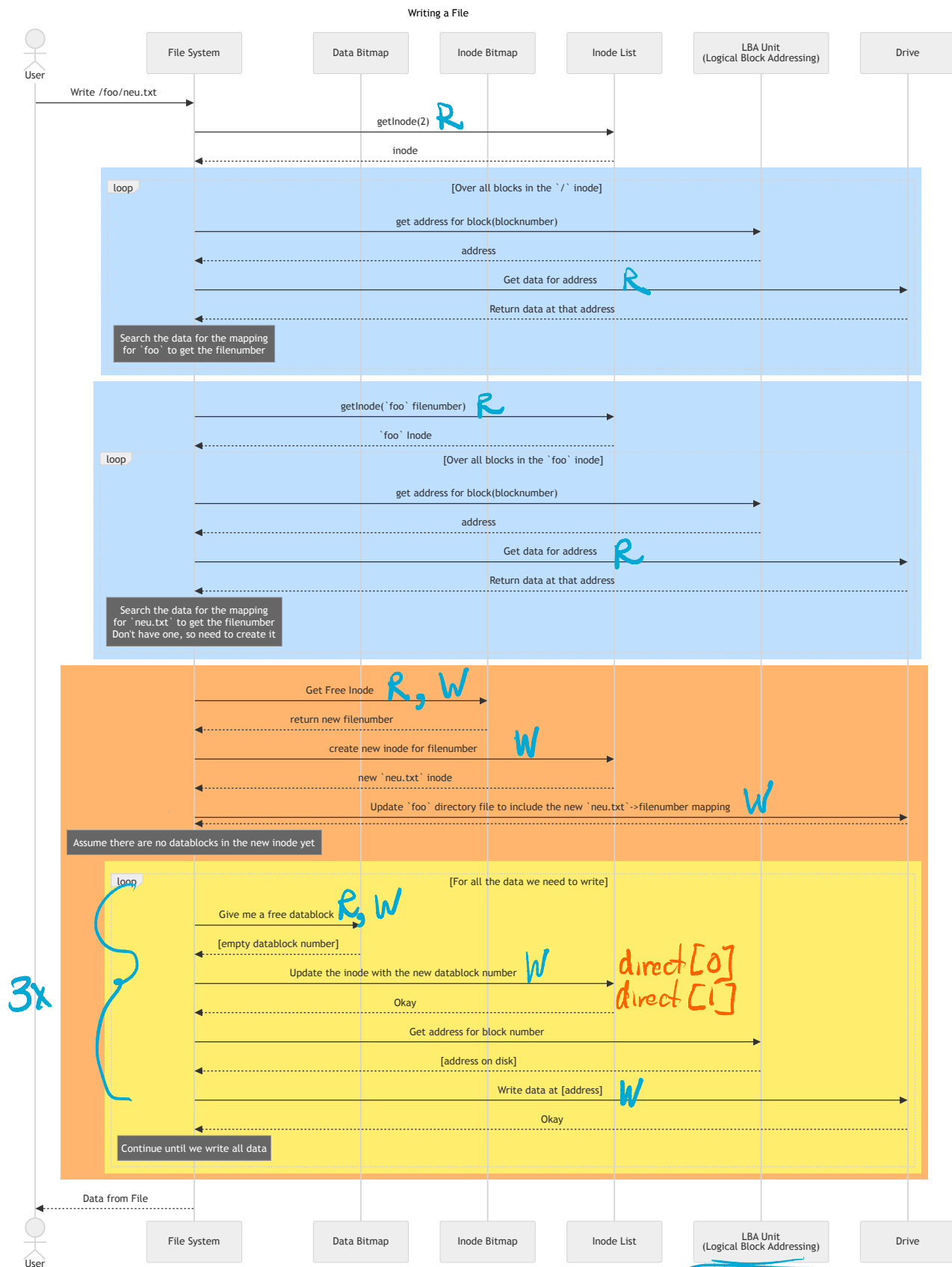
    const char *data = "Hello from neu!\n";
    size_t len = strlen(data);

    size_t written = fwrite(data, 1, len, f);
    if (written != len) {
        perror("fwrite");
        fclose(f);
        return 1;
    }

    fclose(f);
    return 0;
}
```

What the FS is doing:

- Visit the `/` directory
 - Read the `/` inode
 - Find the block that stores the filename/filenumber mapping
 - Read the data in the block and find `foo`'s filenumber
- Visit the `foo` directory
 - Read the `foo` inode
 - Read the data block to find that file `neu` doesn't exist
- Create the `neu` file
 - Find a free inode spot, which will give us our new filenumber
 - Write the metadata for the new inode
 - Write the filename -> filenumber mapping in the data block of the `foo` directory
- Write the data for the `neu` file
 - Read the newly created `neu` inode to find any existing data blocks (which there aren't any)
 - Find a free data block from the data bitmap
 - Allocate/grab that data block, updating the data bitmap
 - Go to that data block on disk, write the bits
 - Update the `neu` inode entry to store the data block
 - Repeat the write steps until all the data is written for the file



Assumptions and simplifications:

- The directory **foo** already exists
- The directory **foo** only has a few entries/it fits in one block
- The file **neu** does not exist

Questions to answer:

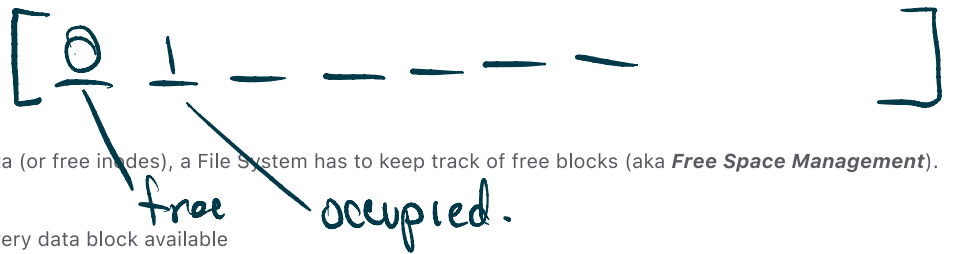
- How many READ operations do we do?

- Are there ever any WRITE operations?
- How many inodes do we visit?

Finding Free Blocks

In order to efficiently find blocks to write data (or free inodes), a File System has to keep track of free blocks (aka **Free Space Management**).

- A very simple approach (used in Lab 5):
 - Use a bitmap that has one entry for every data block available
 - Set a bit to 1 to indicate the corresponding block is used, and 0 to indicate it's free
- A bitmap is an array of `uint32_t` ints
 - Each int has 32 bits, so can keep track of the free/taken status of 32 different blocks
 - If we use 1024 ints to track the block status, that's 32,768 blocks we can keep track of
 - A block holds 4096 bytes
 - We'll make our bitmaps "block-aligned": Each bitmap takes up an entire block
 - Since a block is 4096 bytes, that's 32,768 bits!
- Other approaches (more next week):
 - Free lists
 - A linked list that tracks free blocks
 - B-tree based structures



$[x, y, \dots]$



Bigger Drive
↓
More Blocks
↓
Bigger Data
Bitmap.

Deleting a File: `unlink("/foo/bar")`

Goal: delete the file `/foo/bar`.

In Unix or Linux, the way to do this via C (POSIX):

```
#include <unistd.h>

int result = unlink("/foo/bar");
if (result != 0) {
    perror("Error");
}
```

NOTE: You can also use `remove("/foo/bar.txt")` or `rmdir("/foo")` for directories; both of these calls end up calling `unlink()`.

- `/foo/bar` is a 12 KB file (3 data blocks)
- The inode number is **23**
- it occupies data blocks **32, 33, 34**.

We call `unlink("/foo/bar")`.

Unlike `open()` or `create()`, `unlink()` does **not** return a file descriptor — it just removes the directory entry and, if the link count drops to zero, frees the inode and data blocks. Here's every I/O that happens:

Step 1: Traverse the path (same as open)

The FS has to find `bar`'s inode, which means walking the path from root.

`/foo/bar`

Operation	Structure	Why
read	root inode (inode 2)	find root's data block
read	root directory data	find entry for <code>foo</code> , get inode 44
read	foo's inode (inode 44)	find foo's data block
read	foo's directory data	find entry for <code>bar</code> , get inode 23

Operation	Structure	Why
read	bar's inode (inode 23)	check permissions; read link count

Step 2: Decrement the link count

`bar`'s inode has a `links_count` field. `unlink()` decrements it by 1. If it was 1 (no hard links), it hits 0 — meaning we can free everything. If it were greater than 1 (hard links exist), we'd stop here and only do the directory entry removal below.

Assuming `links_count` drops to 0:

IF link count = 5

Operation	Structure	Why
write	bar's inode (inode 23)	update <code>links_count</code> to 0, set <code>dtime</code>

Step 3: Free the data blocks

The FS clears the bits in the **data bitmap** for blocks 32, 33, and 34.

Operation	Structure	Why
read	data bitmap	load the current bitmap into memory
write	data bitmap	clear bits for blocks 32, 33, 34

Note: the actual data in those blocks is **not zeroed out** — the bits are just flipped to 0 in the bitmap, marking the blocks as available. The old bytes sit there until something else overwrites them (which is why `unlink` is not the same as securely wiping a file).

Step 4: Free the inode

The FS clears bit 23 in the **inode bitmap**.

Operation	Structure	Why
read	inode bitmap	load the current bitmap
write	inode bitmap	clear bit for inode 23

Step 5: Remove the directory entry

The FS writes back to `foo`'s directory data block, removing (or zeroing out) the `bar` entry. In vsfs, this is typically done by marking the entry's inode number as 0, or by adjusting `recLEN` so a neighboring entry absorbs the space.

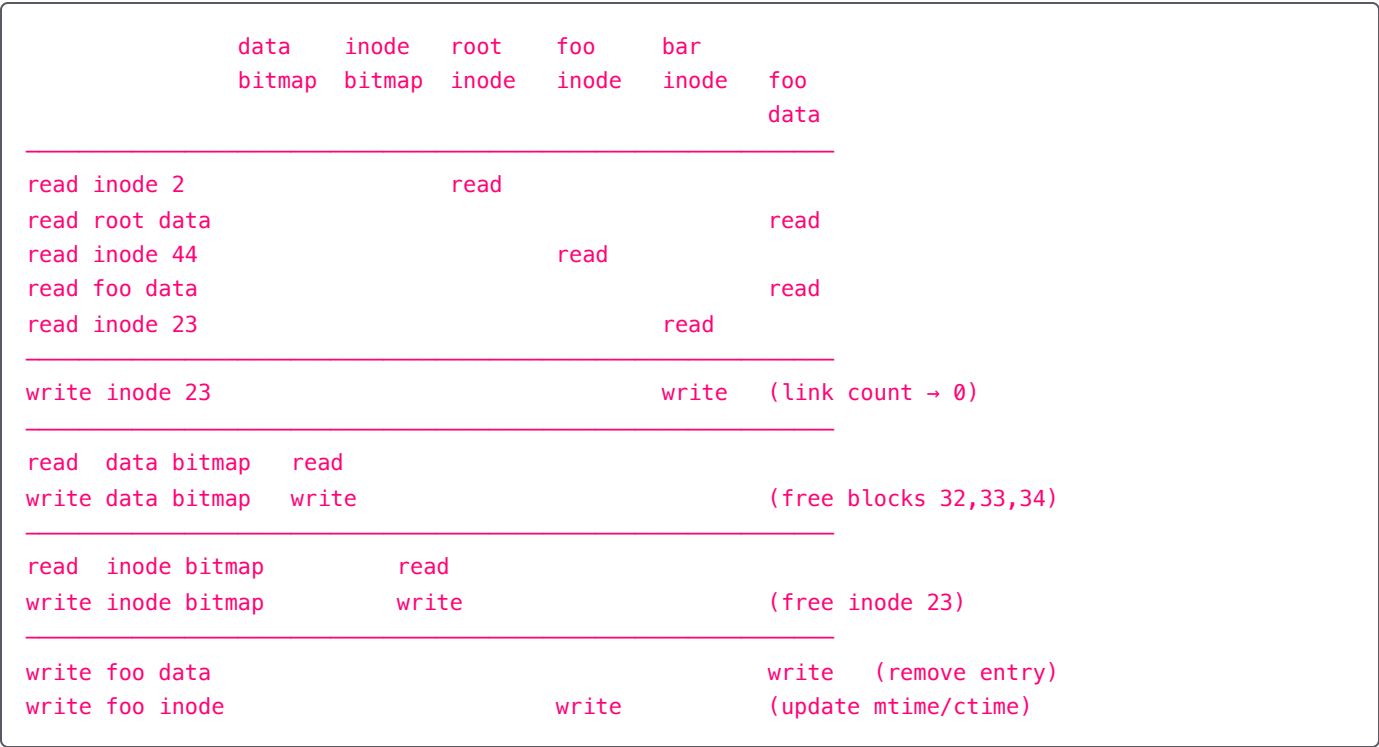
Operation	Structure	Why
write	foo's directory data	remove/zero the entry for <code>bar</code>

The FS also updates `foo`'s inode — the `mtime` and `ctime` fields change because the directory was modified.

Operation	Structure	Why
write	foo's inode (inode 44)	update <code>mtime</code> , <code>ctime</code>

Full I/O timeline

Here's the complete picture in the style of Figure 40.3/40.4 from the chapter (time flows downward):



Total: 5 reads + 6 writes = 11 I/Os for this simple case.

Key things to notice

- **Deletion is not symmetric with creation.** Creating `/foo/bar` and writing 3 blocks cost 10 I/Os. Deleting it costs a similar number — the metadata bookkeeping is comparably expensive even though no user data is written.
- **The data is not erased.** Only the bitmap bits are cleared. The bytes in blocks 32–34 remain intact on disk until reused.

More Details

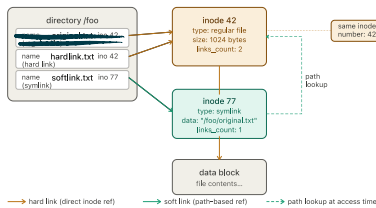
Now that we have a sense for the conceptual components of our file system, let's focus on some implementation details.

- Where do all these components live?
 - On the disk!
 - A simple approach (again, used in Lab 5):
 - Break the disk up into blocks
 - Reserve the first block as the **superblock**— this holds special data for the FS
 - Use the next n blocks for the Block Bitmap
 - How big is n ? Depends on the size of our drive!
 - A Block Bitmap filling up one Block can account for 32,768 blocks (see above), so for every 32,768 blocks our drive can accomodate, we snag another Block to hold a Block Bitmap
 - We store inodes on disk
 - We end up with (basically) the first few blocks taking up our file system metadata, and the rest of the blocks holding the data

Links

Many file systems allow links or aliases to files. Links can be either **hard** or **soft**. Here, we'll introduce the idea to tie back to some of what we talked about, but we'll go into more detail next week.

- In the diagram below, we see that the hard link is mapped to the same inode as the original file.
 - The inode keeps track of how many references there are to that file, and won't delete it as long as any files reference.
- The soft link is mapped to a different inode which links to the original file's inode
 - If the file is deleted, the soft link can be left dangling.



(Note: Image created by Claude)

Tying Everything to Lab 5

- Exercise 2:** Implementing `alloc_diskblock`.
 - This is implementing the data bitmap we talked about in Finding Free Blocks.
- Exercise 3:** Implementing `inode_block_walk` and `inode_get_block`.
 - This is doing the walk stuff we saw in all of our examples.
 - `inode_block_walk` takes in an inode and block id, and you iterate through the direct, indirect and double-indirect blocks to find the right block number (the number of the block the data lives in on disk).
 - Set `ppdiskbno` to point at the location that holds the block number.
 - `inode_get_block` takes in an inode and block id, gets the block number on disk that holds the data, and sets `blk` to point at the data.
 - This will utilize the notion of LBA in our diagrams.
- Exercise 4:** Implementing `inode_truncate_blocks`.
 - This is basically the `delete()` functionality we saw.
- Exercise 5:**
 - This is basically the hard link/soft link/ remove links that we saw.

Next Week: More File Systems

- We looked at an example this week
- Next week we'll look at options for the things we chose
- What could go wrong?
- Tradeoffs

This site uses [Just the Docs](#), a documentation theme for Jekyll.